

REPORT

ORANGE PROBLEM -2

Name : diran somanna CLASS : AIML-G

SRN : PES1UG24AM364

Reference : <https://github.com/sudiksha-chindula/os-u4-orange-problem>

Githublink : <https://github.com/diran07/PES1UG24AM364-pes-vcs>

Screenshots :

1A) test_objects output

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B) object storage

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ s
```

2A) test_tree output

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ make test_tree
make: 'test_tree' is up to date.
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$
```

2B) xxd output

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$
```

3A) pes status

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ ./pes init
echo "hello world" > file1.txt && echo "goodbye" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Segmentation fault (core dumped)
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: f.txt
  untracked: test_objects.c
  untracked: file1.txt
  untracked: pes.h
  untracked: README.md
  untracked: tree.h
  untracked: test_sequence.sh
  untracked: commit.c
  untracked: file2.txt
  untracked: Makefile
  untracked: .gitignore
  untracked: index.c
  untracked: pes.c
  untracked: commit.h
  untracked: tree.c
  untracked: test_tree.c
  untracked: object.c
  untracked: index.h
  untracked: a.txt
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$
```

3B) index file

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776313864 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776313864 6 file2.txt
```

4A) pes log

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ ./pes log
commit 3cf764ffa03de757f314e1d6c1214bbacb44b82c62dc8ef5f4d3fe576c5bac46
Author: PES User <pes@localhost>
Date: 1776314596

Initial commit
```

4B) object growth

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/3c/f764ffa03de757f314e1d6c1214bbacb44b82c62dc8ef5f4d3fe576c5bac46
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/refs/heads/main
```

4C) Screenshot 4C- HEAD

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
3cf764ffa03de757f314e1d6c1214bbacb44b82c62dc8ef5f4d3fe576c5bac46
```

4D) MAKE - INTERGRATION

```
vboxuser@ubuntu:~/PES1UG24AM364-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
test_sequence.sh: line 36: 6912 Segmentation fault      (core dumped) $PES add file.txt hello.txt
make: *** [Makefile:45: test-integration] Error 139
```

5. ANALYSIS QUESTIONS

Q5.1: Implementation of pes checkout <branch>

A branch in the PES-VCS system is represented as a file in `.pes/refs/heads/` that stores the hash of the latest commit. Implementing `pes checkout <branch>` requires updating both repository metadata and the working directory.

Steps Involved

1. Branch Validation

- Check whether `.pes/refs/heads/<branch>` exists
- If not, return an error indicating an invalid branch

2. Read Target Commit

- Read the commit hash stored in the branch file
- This commit represents the snapshot to restore

3. Update HEAD

- Modify `.pes/HEAD` to:

`ref: refs/heads/<branch>`

4. Reconstruct Working Directory

- Read the commit object using the hash
- Extract the corresponding tree object
- Traverse the tree recursively:
 - For each blob → read content and write to working directory
 - For directories → recreate structure recursively
- Remove files in the working directory that are not in the target tree

5. Update Index

- Replace `.pes/index` with entries derived from the target tree

Complexity

This operation is complex due to:

- Safe overwriting/deletion of files
- Recursive traversal of tree structures
- Handling nested directories
- Preventing loss of uncommitted changes

Conclusion:

Checkout is not just a pointer update — it is a full synchronization process between repository state and working directory.

Q5.2: Detecting a Dirty Working Directory

A dirty working directory means there are uncommitted changes in tracked files. Switching branches in this state risks data loss.

Detection Method

1. Compare Working Directory with Index

- For each tracked file:
 - Compare file size and modification time
 - Differences → file is modified

2. Compare Index with Target Branch Tree

- Load the target branch tree
- Compare file hashes with index entries

3. Conflict Detection

A conflict occurs if:

- File is modified in working directory **AND**
- Same file differs in target branch

4. Abort Checkout

- If conflict detected → stop operation with error

Justification

- Index = last staged state
- Object store = committed state

Comparing both ensures detection of unsafe operations and prevents data loss.

Q5.3: Detached HEAD State

A detached HEAD occurs when .pes/HEAD contains a commit hash instead of a branch reference.

Behavior

- New commits can still be created
- These commits are **not linked to any branch**
- They form an independent commit chain

Implications

- Not reachable via branches

- Can be lost if not referenced
- Not visible in normal history traversal

Recovery Mechanisms

1. **Create a New Branch**
 - Point branch to detached commit
2. **Manual Reference Update**
 - Write commit hash into a new ref file
3. **Direct Access**
 - Use commit hash to retrieve data

Conclusion:

Detached HEAD is useful for temporary work but risky if not handled properly.

6. GARBAGE COLLECTION AND SPACE RECLAMATION

Q6.1: Algorithm for Removing Unreachable Objects

Over time, the object store accumulates unused objects. These must be removed to reclaim space.

Algorithm

1. Identify Root References

- Read all branch references in `.git/refs/heads/`
- Each provides a starting commit

2. Traverse Commit Graph

- Use DFS or BFS:
 - Visit commits
 - Add parent commits
 - Add associated trees

3. Traverse Tree Objects

- For each tree:
 - Add blobs and subtrees

4. Mark Reachable Objects

- Store all reachable hashes in a **hash set**

5. Sweep Phase

- Traverse .pes/objects/
- Delete objects not in reachable set

Data Structure

- Hash set $\rightarrow O(1)$ lookup

Complexity

- Must visit all commits, trees, and blobs
 - Efficient structures keep performance acceptable
-

Q6.2: Garbage Collection Race Condition

Running garbage collection concurrently with commits can cause serious corruption.

Race Condition Scenario

1. GC starts scanning
2. New commit is created
3. Commit references new objects
4. GC has not marked them reachable
5. GC deletes them
6. Commit becomes invalid

Consequences

- Repository corruption
- Broken commit history
- Data loss

Preventive Measures

1. **Locking Mechanisms**
 - Prevent concurrent GC and commit
2. **Reference Tracking**
 - Temporarily protect new objects
3. **Two-Phase GC**
 - Mark $\rightarrow$ then delete

4. **Grace Period**

- Delay deletion of new objects

Conclusion

Garbage collection must be carefully synchronized to maintain repository integrity and avoid catastrophic data loss.